Parking Lot System Design

Low Level Design (LLD)

Factory Pattern + Strategy Pattern + SOLID Principles

- ✓ Multiple vehicle types (Car, Bike, Truck)
- ✓ Multiple payment methods (Cash, Card, UPI)
- ✓ Dynamic pricing models
- ✓ Easy future extensions via new classes

Problem Statement

Imagine a mall parking lot. Vehicles enter, get tickets, park, and pay at exit. We need to design this system cleanly.

■ Vehicle Types

Car, Bike, Truck

■ Entry & Exit Gates

Controlled access

■■ Ticket Generation

On vehicle arrival

■ Payment Methods

Cash, Card, UPI

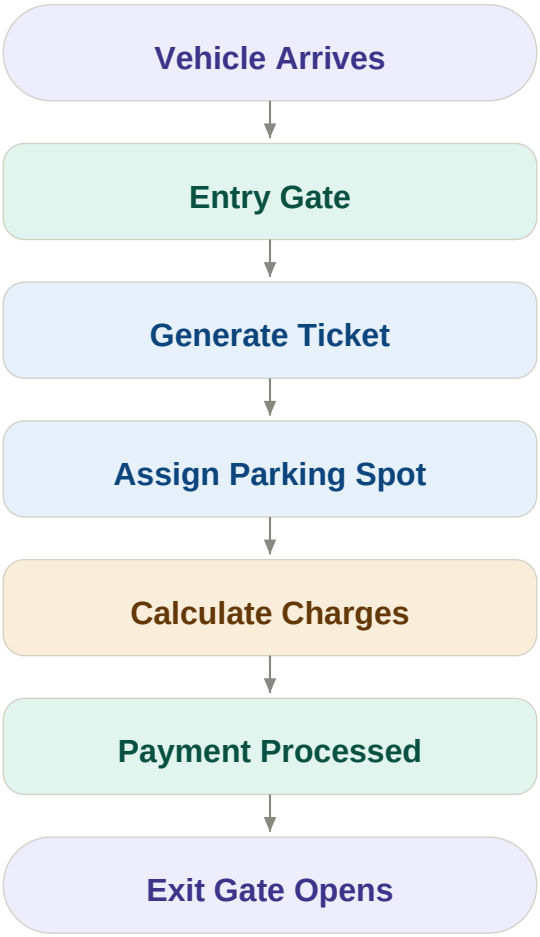
■ Pricing Models

Time / Event based

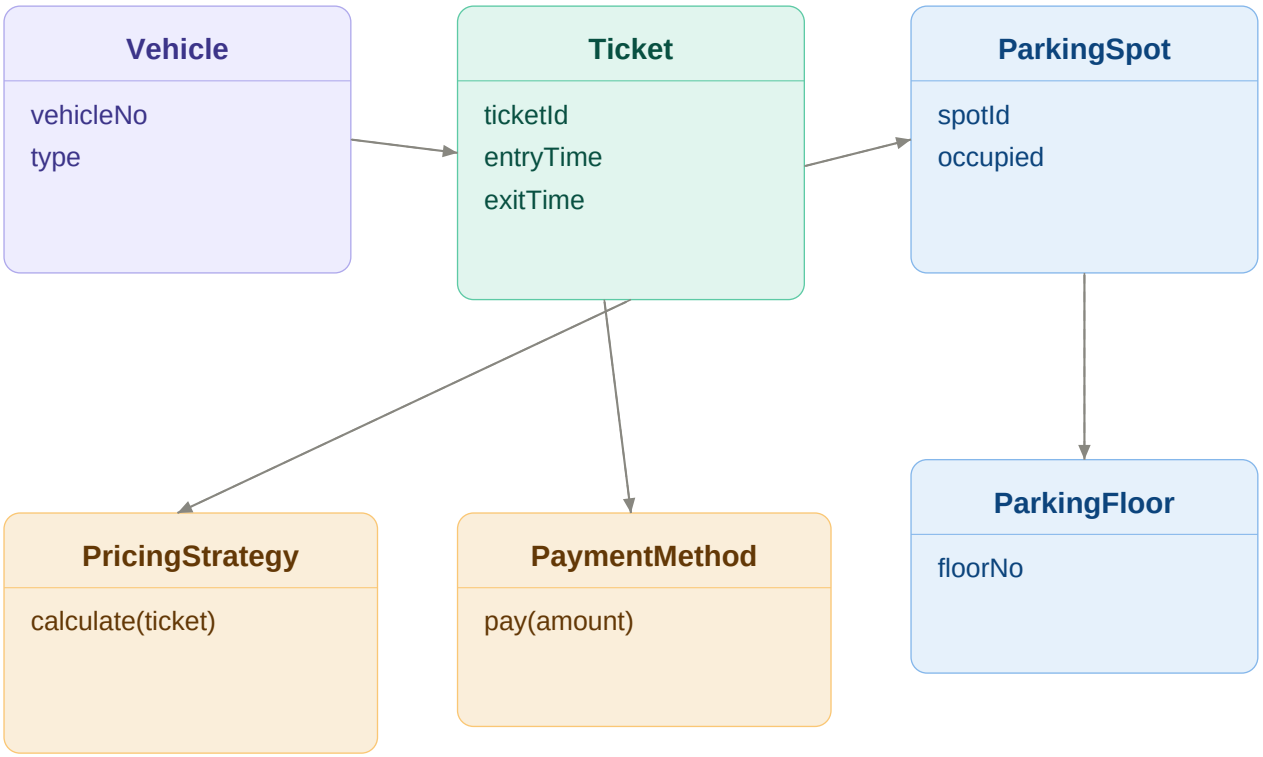
■ Extensibility

Add features easily

High Level Flow



Entity Relationships



Factory Pattern

Centralize Object Creation

Without Factory ✗

```
if(type == CAR)
    return new Car();

if(type == BIKE)
    return new Bike();

if(type == TRUCK)
    return new Truck();

// Modify for every
// new vehicle type!
```

With Factory ✓

```
Vehicle v =
  VehicleFactory
    .createVehicle(type);
```

Factory classes:

- VehicleFactory
- GateFactory
- PaymentFactory
- PricingStrategyFactory

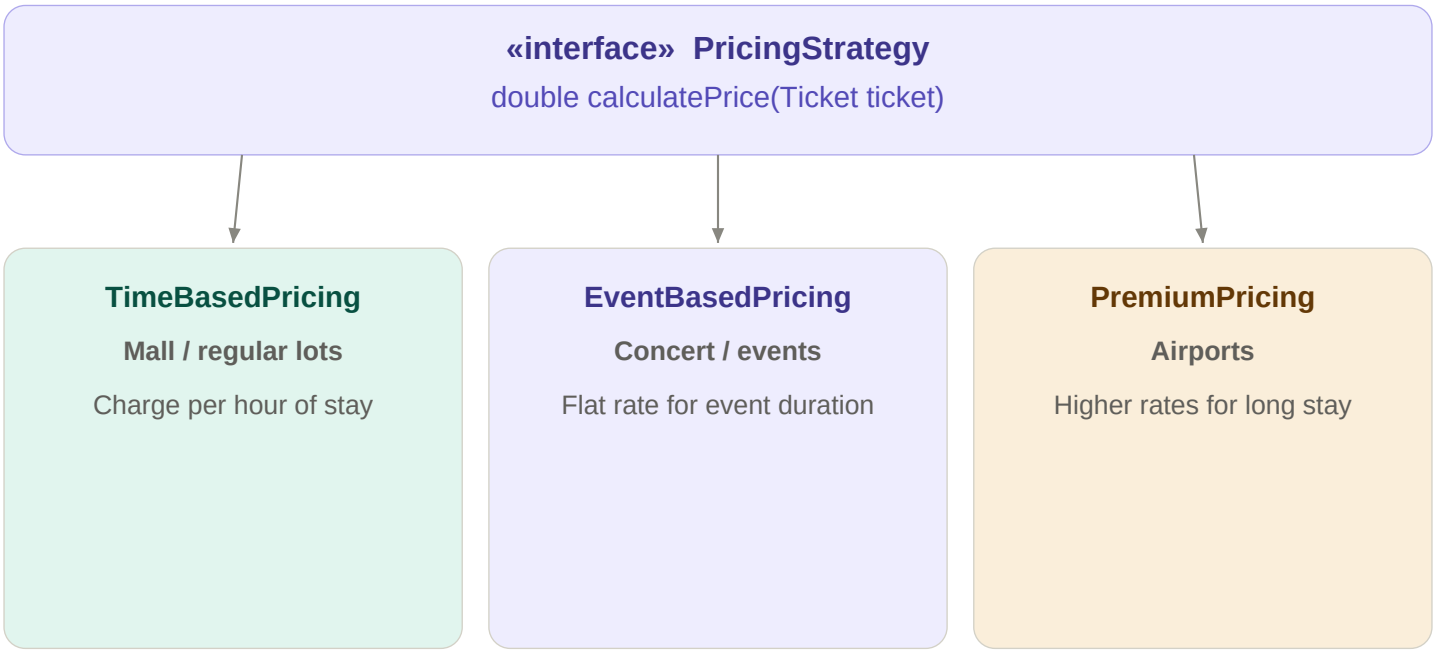
✓ Centralized creation

✓ Easy to extend

✓ Cleaner client code

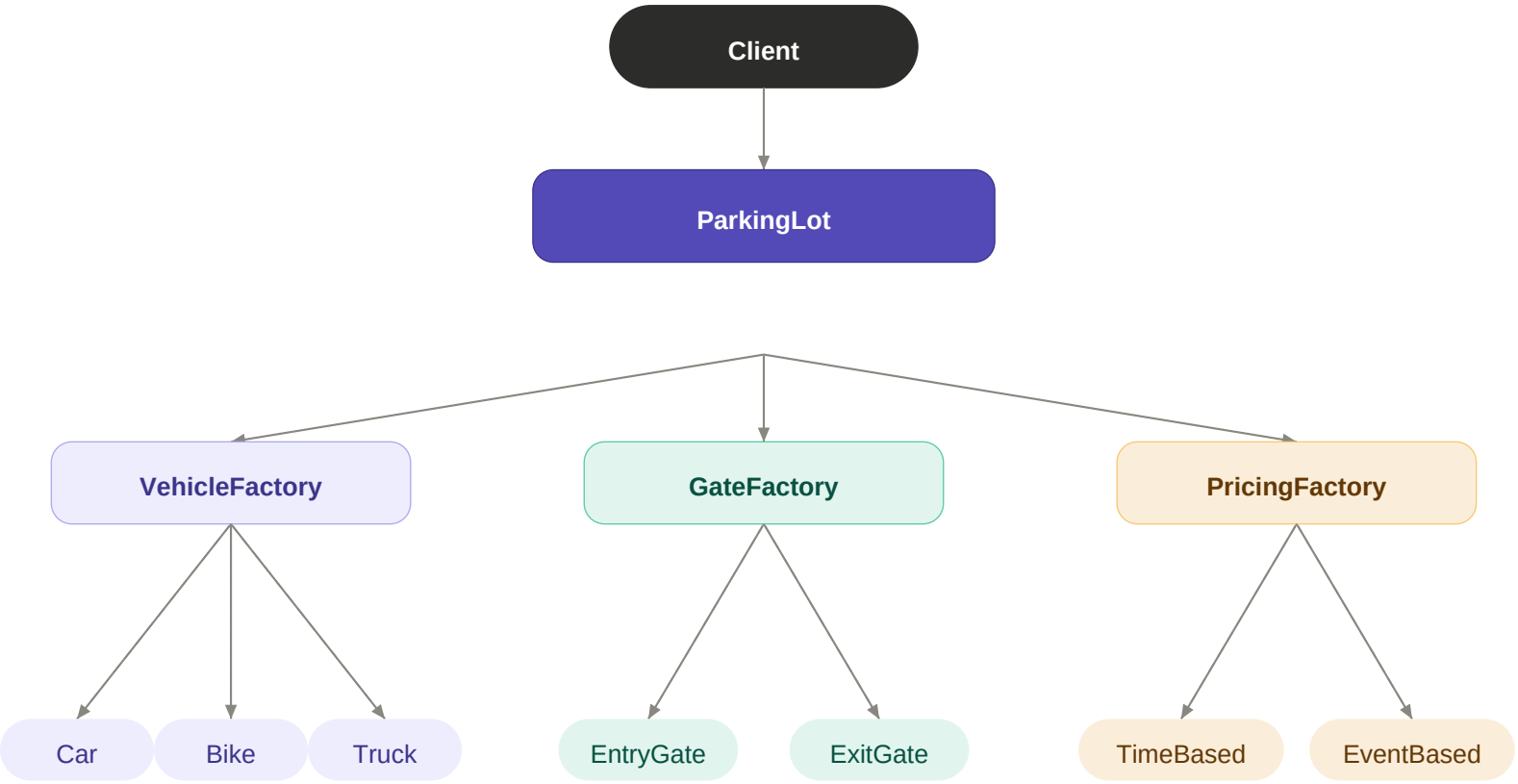
Strategy Pattern

Swap Pricing at Runtime



New pricing rule = new class. Zero existing code changes.

Complete Architecture



Key Learnings

New feature — wrong way

- ✗ Modify existing classes
- ✗ Chain of if-else blocks
- ✗ Risky regressions
- ✗ Tightly coupled code

New feature — right way

- ✓ Create a new class
- ✓ Implement the interface
- ✓ Zero existing code changes
- ✓ SOLID principles applied

Extend without touching existing code:

UPI Payment | EV Charging | VIP Parking | Weekend Pricing | Reservations | Festival Pricing

LinkedIn Post Caption

■ Built a Parking Lot System (LLD) in Java

While most Parking Lot implementations focus only on vehicle entry and exit, I wanted to design a system that remains scalable when new business requirements arrive.

■ What I Implemented

- ■ Vehicle Types (Car, Bike, Truck)
- ■ Entry & Exit Gates
- ■ Ticket Generation
- ■ Parking Floors & Spots
- ■ Multiple Payment Methods
- ■ Dynamic Pricing Models

■ Factory Pattern → Object Creation

- ■ VehicleFactory
- ■ GateFactory
- ■ PaymentFactory
- ■ PricingStrategyFactory

■ Strategy Pattern → Runtime Behavior

- ■ TimeBasedPricing
- ■ EventBasedPricing
- ■ CashPayment
- ■ CardPayment

■ Biggest Takeaway

Good LLD means adding features like UPI Payments, EV Charging, VIP Parking or Reservation System requires creating new classes — not modifying existing ones. That's where Factory, Strategy, and SOLID start paying off.

#Java #SystemDesign #LLD #DesignPatterns #FactoryPattern #StrategyPattern #SOLID #SoftwareEngineering #BackendDevelopment #JavaDeveloper